

RELATÓRIO FINAL – COMPILADOR PYTHON DIDÁTICO

Marcelo Gabriel Euclides Da Silva – Matrícula 01605103

Giorgio Rafael Ataide Guimarães – Matrícula 01620989

Luiz Gustavo Freire Barbosa – Matrícula 01864307

Matheus Wendell de Paula Souza – Matrícula 01602509

Guilherme Oliveira Ribeiro – Matrícula: 01606196

Pedro Henrique Alves Lessa – 01638620

Matheus Venâncio Da Silva – 01628436

Yuri Mascarenhas – 01327348

Felipe de Lima Passos – 01647492

Maria Eduarda Cardoso Ferreira – 01724547

Maceió – AL
2026

Disciplina: Compiladores

Curso: Ciência da Computação

1 Visão Geral do Projeto

1.1 Problema a ser resolvido

O estudo de compiladores é um dos temas mais importantes da Ciência da Computação, pois permite compreender como linguagens de programação são interpretadas e executadas pelos computadores. Entretanto, muitos estudantes encontram dificuldades para visualizar na prática as etapas envolvidas nesse processo, uma vez que grande parte do conteúdo é apresentada de forma teórica.

Diante desse cenário, foi desenvolvido um compilador didático inspirado na linguagem Python, permitindo que o usuário escreva código-fonte, realize sua análise e visualize os resultados da execução em uma interface gráfica intuitiva.

1.2 Público-alvo

O sistema foi desenvolvido para estudantes da disciplina de Compiladores, professores da área de Linguagens Formais e Compiladores, além de desenvolvedores iniciantes interessados em compreender o funcionamento interno de uma linguagem de programação.

1.3 Proposta de Valor

A proposta do projeto consiste em fornecer uma ferramenta educacional capaz de demonstrar, de forma prática, as principais etapas de um compilador moderno. O sistema realiza análise léxica, análise sintática e interpretação de código, permitindo ao usuário visualizar erros, executar programas e acompanhar o estado das variáveis durante a execução.

2 Requisitos do Sistema

2.1 Requisitos Funcionais (RF)

- RF01: Permitir ao usuário escrever código-fonte em um editor integrado.
- RF02: Realizar análise léxica do código inserido.
- RF03: Realizar análise sintática utilizando uma gramática inspirada na linguagem Python.
- RF04: Executar programas válidos por meio de um interpretador interno.
- RF05: Exibir mensagens de erro contendo linha e coluna da ocorrência.
- RF06: Permitir a utilização de estruturas condicionais.
- RF07: Permitir a utilização de estruturas de repetição.
- RF08: Exibir as variáveis armazenadas em memória após a execução.

2.2 Requisitos Não Funcionais (RNF)

- RNF01: O sistema deve ser desenvolvido utilizando JavaScript.
- RNF02: A interface gráfica deve ser desenvolvida utilizando React.
- RNF03: A aplicação deve possuir interface responsiva.
- RNF04: O sistema deve impedir execuções infinitas por meio de controle de passos.
- RNF05: A aplicação deve fornecer feedback visual sobre o resultado da compilação.

3 Regras de Negócio

- Variáveis devem existir antes de serem utilizadas.
- Operações matemáticas devem ocorrer apenas entre valores numéricos compatíveis.
- Concatenação de textos deve ocorrer apenas entre valores textuais.
- O comando `range()` deve receber apenas valores inteiros.
- Loops infinitos devem ser interrompidos automaticamente quando o limite de execução for atingido.
- Comparações devem ocorrer apenas entre tipos compatíveis.

4 Arquitetura do Sistema

O sistema foi desenvolvido seguindo uma arquitetura modular, composta pelas etapas de análise léxica, análise sintática e interpretação de código.

4.1 Fluxo de execução

A interface desenvolvida em React é responsável pela entrada do código-fonte e pela exibição dos resultados. O módulo **Lexer** realiza a análise léxica, o **Parser** constrói a árvore sintática e o **Interpreter** executa as instruções geradas.

5 Análise Léxica

A análise léxica é a primeira etapa do processo de compilação. Seu objetivo é converter o código-fonte em uma sequência de tokens compreensíveis pelo compilador.

Entre os tokens suportados estão:

Código	Token
if	IF
else	ELSE
while	WHILE
print	PRINT
true	BOOLEAN

5.1 Exemplo

Código-fonte:

```
x = 10
```

Tokens gerados:

```
IDENTIFIER EQUAL NUMBER
```

Além disso, o analisador léxico reconhece operadores matemáticos, operadores lógicos, operadores relacionais, números inteiros, números decimais, strings, caracteres e estruturas de indentação inspiradas na linguagem Python.